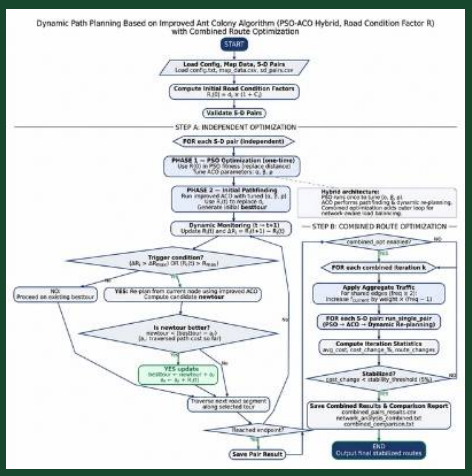
Thesis Flowchart Explanation



df59d093-f9b8-4ddb-86cb-9bef82c21302.docx
Document

this is word file and my updated flwo hcart of my thesis and now you have to explin me these thing . Do you understand ?

Yes 👍 I understand your request.

You shared:

1. **Your thesis flowchart (image)**
2. **Your Word document with methodology, tables, and results**

You want **a clear explanation of the flowchart and the updated algorithm** so you can **understand it well (for thesis writing or defense).**

I will explain it **step-by-step in simple terms** so you can easily present it.

---

1. Overall Idea of Your Algorithm

Your research proposes a **Dynamic Path Planning Algorithm** using a **PSO–ACO Hybrid**.

Goal:
👉 Find the **best routes between multiple source-destination pairs** while considering **traffic congestion dynamically**.

Main concepts:

- **PSO (Particle Swarm Optimization)** → finds the best parameters for ACO.
- **ACO (Ant Colony Optimization)** → finds the best path.
- **Road condition factor R** → represents traffic condition.
- **Dynamic monitoring** → detects congestion and re-plans routes.
- **Combined optimization** → considers congestion from multiple routes using the same roads.

So your system works in **two levels**:

1️⃣ **Single pair optimization**
2️⃣ **Network level optimization**

---

2. Input Stage (Start of Flowchart)

The algorithm first loads the required data.

Step 1: Load Configuration

Inputs:

- `config.txt`
- `map_data.csv`
- `sd_pairs.csv`

These contain:

• Road network
• Edge distances
• Node connections
• Source-Destination pairs (17 pairs)

Example pair:

```
0 → 9
4 → 7
5 → 9
```

Your updated algorithm processes **all 17 pairs**.

df59d093-f9b8-4ddb-86cb-9bef82c…

Step 2: Compute Initial Road Condition Factor

Each road gets a **road condition value R(i)**.

Formula:

$$R(i) = d_i \times (1 + C_i)$$

Where:

- **d_i** = road distance
- **C_i** = congestion factor

Meaning:

➡️ Longer or congested roads become **less attractive**.

Step 3: Validate S-D pairs

The algorithm checks if:

- Source node exists
- Destination node exists
- Path is possible

Then optimization starts.

---

3. STEP A — Independent Optimization (for each S-D pair)

This part runs **independently for each pair**.

Example:

```
Pair P0 → 0→9
Pair P1 → 0→7
Pair P2 → 0→3
```

Your system processes **17 pairs individually**.

df59d093-f9b8-4ddb-86cb-9bef82c…

---

4. Phase 1 — PSO Optimization

PSO is used **only once** to tune ACO parameters.

PSO finds the best:

```
α  (pheromone importance)
β  (heuristic importance)
ρ  (evaporation rate)
```

PSO parameters:

Parameter	Value
Swarm size	30
Iterations	100
w	0.7
c1	1.5
c2	1.5

PSO searches parameter ranges:

```
α → 0.5 to 1.5
β → 1 to 5
ρ → 0.5 to 1
```

Purpose:

👉 Improve ACO performance.

---

5. Phase 2 — Initial Pathfinding (ACO)

Using optimized parameters from PSO.

ACO process:

1️⃣ Ants explore the network
2️⃣ Pheromone trails guide path selection
3️⃣ Shorter and better roads get more pheromone

Output:

```
Initial best path
```

Example:

```
0 → 1 → 5 → 9
```

---

6. Dynamic Monitoring (Traffic Simulation)

Now the algorithm simulates **traffic over time**.

At each step:

```
t → t+1
```

Update:

```
R_i(t)
ΔR_i = R_i(t+1) - R_i(t)
```

Meaning:

Road conditions **change over time**.

---

7. Congestion Detection

The system checks if congestion occurs.

Trigger conditions:

```
ΔR_i > ΔRmax
OR
R_i > Rmax
```

Values used:

```
Rmax = 150
ΔRmax = 5
```

If NO congestion

The vehicle continues on the same route.

If YES congestion

The system performs:

```
Dynamic Re-planning
```

Steps:

1️⃣ Run **Improved ACO again**
2️⃣ Generate **candidate new route**

Example:

Old route:

```
3 → 2 → 6 → 9
```

New route:

```
6 → 9
```

---

8. Route Comparison

The algorithm compares:

```
new route cost
vs
old route cost
```

Formula:

```
cost = besttour − a
```

If the **new route is better**:

```
Update best route
```

Otherwise:

```
Keep old route
```

---

9. Continue Until Destination

The algorithm repeats this until:

```
Reached endpoint
```

Then result is saved.

Example result:

Pair	Path
P10	5 → 9

Your best result:

```
Cost = 194.94
```

Lowest among all routes.

df59d093-f9b8-4ddb-86cb-9bef82c…

---

10. STEP B — Combined Route Optimization

This is the **new contribution of your thesis**.

Problem with independent optimization:

Multiple routes may use the **same road**.

Example:

```
5 → 9 used by 5 routes
```

This causes congestion.

Edge **5→9** was the biggest bottleneck.

df59d093-f9b8-4ddb-86cb-9bef82c…

Shared Edge Detection

The algorithm calculates:

```
freq_i = number of routes using edge i
```

Example:

```
Edge 5 → 9
freq = 5
```

Apply Traffic Penalty

Formula:

$$C_i(t) += 0.3 \times (freq_i - 1)$$

Meaning:

If many routes use the same road → **penalty increases**.

This simulates **traffic congestion**.

---

11. Re-Optimize All Pairs

After penalties are applied:

The algorithm **re-runs the optimization for all pairs**.

This loop continues until convergence.

Stopping conditions:

```
Average cost change < 5%
OR
Max iterations = 5
```

In your experiment:

```
Converged after 3 iterations
```

Cost progression:

```
356.60
387.43
370.40
```

Final change:

```
4.39%
```

Below the threshold.

df59d093-f9b8-4ddb-86cb-9bef82c…

---

12. Network Analysis

After stabilization, the system analyzes:

Edge usage

Which roads are most used.

Example:

```
5 → 9  (5 routes)
```

Node usage

Most critical intersections.

Example:

```
Node 5 used by 10 routes
```

These are called **network bottlenecks**.

df59d093-f9b8-4ddb-86cb-9bef82c…

---

13. Output Files

The algorithm generates:

File	Content
all_pairs_results.csv	results of each pair
combined_results.csv	final combined results
network_analysis.txt	bottleneck analysis
comparison.txt	independent vs combined

---

14. Final Output

The algorithm outputs:

```
Final stabilized routes
```

Performance summary:

Metric	Value
Success rate	100%
Pairs	17
Average cost (independent)	357.89
Average cost (combined)	370.40

Combined optimization slightly increases cost but produces **more realistic traffic-aware routes**.

df59d093-f9b8-4ddb-86cb-9bef82c…

---

15. Your Main Thesis Contributions

Your research adds **three main contributions**:

1️⃣ All-Pairs Optimization

Instead of one pair → **17 S-D pairs**

2️⃣ Combined Route Optimization

Traffic penalties for shared edges.

3️⃣ Network Bottleneck Analysis

Detect critical roads and intersections.

---

16. Simple One-Line Summary (For Viva)

You can say:

> "The proposed system integrates PSO-optimized ACO with dynamic congestion monitoring and a combined network optimization loop to generate realistic traffic-aware routes for multiple source-destination pairs."

✅ If you want, I can also explain:

- **How to present this flowchart in thesis defense**
- **How to explain PSO-ACO hybrid in viva**
- **How to write the methodology chapter from this flowchart**
- **How to convert this into pseudo-code**

Just tell me 👍

Sources

Step 3: Validate S-D pairs

The algorithm checks if:

Source node exists

Destination node exists

Path is possible at which step sd pair validation is executed ? in the updated flow chart tellmme

In your **updated flowchart**, the **S–D pair validation happens very early**, before the optimization begins.

Look at the **top section of the flowchart**. The sequence is:

1️⃣ **Load Config, Map Data, S-D Pairs**
2️⃣ **Compute Initial Road Condition Factors**
3️⃣ **Validate S-D Pairs**
4️⃣ **STEP A: Independent Optimization – For each S-D pair**

So the **validation step is executed right after computing the road condition factors and before the algorithm enters the optimization loop.**

Exact Position in Your Flowchart

The order is:

```
START
   ↓
Load Config, Map Data, S-D Pairs
   ↓
Compute Initial Road Condition Factors
   ↓
Validate S-D Pairs   ← (Validation occurs here)
   ↓
STEP A: Independent Optimization
FOR each S-D pair
```

So **S-D validation is done once at the beginning**, not inside the optimization loop.

Why Validation Happens Here

This step ensures the algorithm does not try to optimize invalid pairs.

The system checks:

1️⃣ Source node exists

Example
If graph nodes are:

```
0,1,2,3,4,5,6,7,8,9
```

Then:

```
Source = 11 ❌ invalid
```

2️⃣ Destination node exists

Example:

```
Destination = 15 ❌ invalid
```

3️⃣ Path connectivity exists

The algorithm checks if a **path exists between source and destination** in the road network graph.

Example:

```
0 → 9  ✔ valid
```

But if nodes are disconnected:

```
3 → 15  ❌ impossible
```

What Happens After Validation

If a pair is **valid**, it proceeds to:

```
STEP A: Independent Optimization
```

Where the algorithm runs:

```
PSO tuning → ACO pathfinding → Dynamic monitoring
```

Important Point for Your Thesis

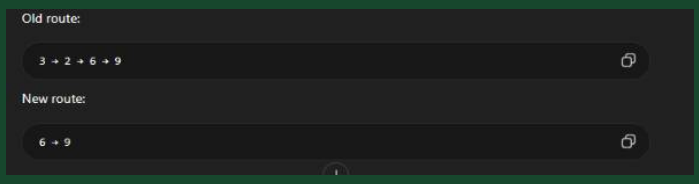
You can write it like this:

> After loading the network configuration and computing the initial road condition factors, all source–destination pairs are validated to ensure that both nodes exist in the graph and that a feasible path between them is possible. Only valid S–D pairs proceed to the independent optimization stage.

✅ Short answer for viva:

> The S-D pair validation is executed **after computing initial road condition factors and before Step A (Independent Optimization)** in the updated flowchart.

If you want, I can also show you **3 important questions examiners usually ask about this flowchart** during a **thesis defense**.



why new route is 6->9 instead of the whole 3->2->6->6->9 tell me

Good question 👍 — this is related to **how your dynamic re-planning works in the flowchart**.

You asked:

> Why does the new route become **6 → 9** instead of **3 → 2 → 6 → 9**?

The key reason is that **re-planning happens from the current node, not from the original source**.

Let's explain it clearly.

---

1. What the Old Route Was

Initial path found by ACO:

```
3 → 2 → 6 → 9
```

This means:

- Source = **3**
- Destination = **9**

---

2. Vehicle Starts Traversing the Route

During simulation the vehicle moves step-by-step:

Step 1

```
3 → 2
```

Step 2

```
2 → 6
```

Now the vehicle is **already at node 6**.

So the traveled part is:

```
3 → 2 → 6
```

---

3. Congestion is Detected

At node **6**, the algorithm checks:

```
ΔRi > ΔRmax
OR
Ri > Rmax
```

If congestion is detected, the algorithm triggers:

```
Re-plan from current node
```

This is shown in the flowchart box:

"Re-plan from current node using improved ACO"

---

4. New Path is Computed From Current Node

Now the new optimization problem becomes:

```
Source = 6
Destination = 9
```

NOT

```
Source = 3
```

So ACO searches the best path from **6 → 9**.

The shortest path is:

```
6 → 9
```

---

5. Final Route Construction

The full route becomes:

```
3 → 2 → 6 + (new route)
```

So the complete path is actually:

```
3 → 2 → 6 → 9
```

But the **replanned segment** is only:

```
6 → 9
```

That is why the table shows:

```
New route: 6 → 9
```

It represents **only the updated segment**, not the whole trip.

---

6. Why This Is Done (Important Concept)

This approach is called:

Local Dynamic Re-planning

Advantages:

✔️ Faster computation
✔️ No need to recompute entire route
✔️ More realistic traffic behavior
✔️ Works like real GPS systems

Example:

Google Maps also recalculates **from your current location**, not from the original starting point.

---

7. Simple Example

Imagine driving:

```
Home → A → B → C → Office
```

You reach **B**, then traffic happens.

Navigation recalculates:

```
B → Office
```

Not

```
Home → Office
```

---

8. How You Can Explain in Thesis/Viva

You can say:

> When congestion is detected during traversal, the algorithm performs dynamic re-planning from the current node rather than the original source. Therefore, only the remaining segment of the path is recomputed. In this case, once the vehicle reaches node 6, the algorithm calculates a new optimal path from 6 to 9, resulting in the updated route segment 6→9.

✅ **Short answer for viva:**

> The new route is **6→9** because re-planning occurs from the **current node (6)** after congestion is detected, not from the original source (3).

If you want, I can also explain **one very important hidden concept in your algorithm** that examiners often ask about:

"Why does some final cost become higher even after optimization?"

(This appears in your results table and many students cannot answer it.)

WHAT WAS UPDATED

Previous work implemented the PSO-ACO dynamic path planner for a single source-destination (S-D) pair. The algorithm used PSO to tune ACO parameters (α, β, ρ), generated an initial best path, and dynamically re-planned when congestion thresholds (R_{max}, ΔR_{max}) were exceeded during simulation. This update extends that framework with three additions:

1.All-pairs routing: The algorithm now processes all 17 source-destination pairs from an input file (sd_pairs.csv), running independent PSO-ACO optimization and dynamic simulation for each pair.

2.Combined route optimization: An outer loop detects shared edges across routes, applies aggregate traffic penalties, and re-solves all pairs iteratively until costs stabilize (change < 5%). This captures real inter-route congestion that independent optimization ignores.

3.Network-level analysis: Edge and node usage frequencies are computed across all routes to identify congestion bottlenecks and critical intersections in the shared network.

can you tell me this part short

Here is a **short and clear version** of the highlighted part:

Combined Route Optimization (short explanation):

> An outer loop identifies roads that are used by multiple routes, applies traffic penalties to those shared edges, and re-optimizes all source–destination pairs repeatedly until the route

> costs stabilize (cost change < 5%). This models real traffic congestion caused by multiple vehicles using the same roads.

Even shorter (1–2 lines for presentation):

> The algorithm detects shared roads among routes, adds congestion penalties, and repeatedly re-optimizes all routes until the overall cost stabilizes.

If you want, I can also give you a **very strong 1-sentence version suitable for thesis defense slides.**

> why it take 17 src ->destination pair...but i have to take all src->des pair...and as of myknowledge combination of (all 0to 9 has many more pairs) can you explian me in hsort and logically

Good observation 👍 This is a **very logical question**.

You are thinking:

> If the network has nodes **0–9**, then all possible source-destination combinations should be much more than **17 pairs**, right?

Yes, mathematically you are correct. Let's see why **17 pairs** were used.

---

1. All Possible S-D Pairs in a 10 Node Network

If nodes are:

```
0,1,2,3,4,5,6,7,8,9
```

Total nodes = **10**

Possible **directed S-D pairs**:

$$N(N-1)$$

So:

$$10 \times 9 = 90$$

So **90 possible pairs exist**.

Example:

```
0→1
0→2
0→3
...
9→8
```

---

2. Why Your Algorithm Uses Only 17 Pairs

Because the **input file sd_pairs.csv contains only 17 selected pairs**.

So the algorithm processes **only the pairs provided in the dataset**, not all mathematical combinations.

This is common in traffic simulations.

Reasons:

1️⃣ Realistic traffic demand

In real cities, **not every node sends traffic to every other node**.

Example:

```
Home → Office
Station → Airport
School → Market
```

So only **important routes** are selected.

2️⃣ Computational complexity

If you used all pairs:

```
90 pairs
```

Then every combined optimization iteration would solve **90 routing problems**, which is very expensive.

Using **17 representative pairs** makes the simulation manageable.

3️⃣ Benchmark experiment design

Researchers usually select **a subset of pairs** to represent:

- major flows
- realistic traffic demand

---

3. Logical Explanation (Short)

You can explain it like this:

> Although a 10-node network can theoretically produce 90 source-destination combinations, the algorithm processes only the 17 S-D pairs defined in the input file. These pairs represent selected traffic demands in the network and reduce computational complexity while still capturing realistic congestion interactions.

---

4. Very Short Answer (For Viva)

> The algorithm uses 17 S-D pairs because they represent selected traffic demand scenarios from the input dataset. Processing all possible node combinations would significantly increase computational complexity without adding meaningful traffic patterns.

✅ If you want, I can also show you **one important mistake students make in thesis defense when explaining "All-Pairs Routing"**, which examiners often catch.

METHODOLOGY — WHAT WAS ADDED

A. All-Pairs Routing Loop (Step A)

The original three-phase algorithm (PSO tuning → ACO pathfinding → dynamic simulation) is now executed independently for every S-D pair in sd_pairs.csv. Each pair has a separate PSO run and ACO execution. Results — initial path, final path, cost, and reroute events — are saved to all_pairs_results.csv.

B. Combined Route Optimization Loop (Step B)

After all pairs are solved independently, the combined loop detects edges shared by 2 or more routes and penalizes their congestion:

$$C_i(t) \mathrel{+}= 0.3 \times (freq_i - 1) \quad \text{for edges with } freq_i \geq 2$$

Updated road condition factors R_i(t) are recomputed, and all pairs are re-solved. The loop continues until the average cost change drops below 5% or 5 iterations complete. This simulates the real-world effect of multiple vehicles competing for the same road segments.

C. Network Analysis

Edge and node usage frequencies are computed across all stabilized routes. Edges used by ≥ 2 routes and nodes appearing in ≥ 3 routes are flagged as bottlenecks. These are exported to network_analysis.txt and network_analysis_combined.txt.

D. Algorithm Parameters

All parameters remain identical to the single-pair implementation (see Table I). No changes were made to PSO/ACO core logic — only the outer structure was extended.

tell me it in in short and logically

Here is a **short and logical explanation of the whole section**:

A. All-Pairs Routing Loop (Step A)

The PSO-ACO algorithm (PSO tuning → ACO pathfinding → dynamic simulation) is run **separately for each S-D pair** from `sd_pairs.csv`. For every pair, the algorithm generates an **initial path, final path, cost, and reroute events**, which are saved in `all_pairs_results.csv`.

B. Combined Route Optimization (Step B)

After solving all pairs independently, the algorithm **detects roads shared by multiple routes**. It adds **traffic penalties to those edges**, updates road conditions, and **re-optimizes all pairs iteratively** until the route costs stabilize (change < 5% or max 5 iterations). This models real traffic where multiple vehicles use the same roads.

C. Network Analysis

The algorithm analyzes **how often edges and nodes are used** across all routes. Roads used by **≥2 routes** and nodes appearing in **≥3 routes** are identified as **traffic bottlenecks** and saved for analysis.

D. Algorithm Parameters

All **PSO and ACO parameters remain the same** as the original single-pair algorithm. Only the **outer structure (multi-pair and combined optimization loops)** was added.

✅ **Very short summary (2 lines):**

> The algorithm first optimizes each source-destination pair independently using PSO-ACO.
> Then it performs a combined optimization that penalizes shared roads, re-optimizes routes until costs stabilize, and analyzes the network to detect congestion bottlenecks.

Each pair has a separate PSO run and ACO execution. Results — initial path, final path, cost, and reroute events — are saved to all_pairs_results.csv. hwat is menat by inital na dfinal path here tell me

Good question 👍 This is an **important concept in your algorithm**.

Meaning of Initial Path

The **initial path** is the **first best route found by ACO** after PSO tunes the parameters.

Process:

```
PSO → finds best α, β, ρ
ACO → finds the first optimal path
```

This path is calculated **before any traffic simulation or congestion occurs**.

Example:

```
Initial path: 3 → 2 → 6 → 9
```

It is the **original planned route**.

Meaning of Final Path

The **final path** is the **route after dynamic simulation and possible rerouting**.

During simulation:

- Road conditions change
- Congestion may occur
- The algorithm may **re-plan the route**

So the final path may be **different from the initial one**.

Example:

```
Initial path: 3 → 2 → 6 → 9
Final path:   6 → 9
```

This means the algorithm **rerouted after reaching node 6** because of congestion.

Simple Logical Difference

Term	Meaning
Initial Path	First route found by ACO before traffic changes
Final Path	Route after dynamic monitoring and rerouting

One-line answer (for thesis/viva)

> The initial path is the first optimal route generated by the ACO algorithm, while the final path is the route after dynamic simulation where congestion may trigger rerouting.

If you want, I can also explain **one hidden thing about your results table that many students misunderstand about "final path"**.

TABLE I – Algorithm Configuration Parameters

Parameter	Value	Description
PSO Swarm / Iterations	30 / 100	Particle count & search steps
w / c1 / c2	0.7 / 1.5 / 1.5	Inertia, cognitive, social weights
α / β / ρ range	[0.5,1.5]/[1,5]/[0.5,1]	ACO parameter search bounds
NUM_ANTS / ACO_ITERS	10 / 100	Colony size & iterations
R_MAX / ΔR_MAX	150.0 / 5.0	Congestion re-plan thresholds
SIM_STEPS	20	Dynamic simulation steps per pair
COMBINED_ITERS / Threshold	5 / 5%	Outer loop max iters & stop criterion

can you explain me this term logically and i short ...waht it means /

Here is a **short and logical explanation of each parameter in your table**.

1️⃣ PSO Swarm / Iterations (30 / 100)

- **30 particles** search for the best solution.
- The search runs for **100 iterations**.
  👉 Used to find the best **ACO parameters (α, β, ρ)**.

2️⃣ w / c1 / c2 (0.7 / 1.5 / 1.5)

These control **PSO particle movement**.

- **w (0.7)** → inertia (keeps current movement)
- **c1 (1.5)** → cognitive factor (particle learns from itself)
- **c2 (1.5)** → social factor (particle learns from swarm)

👉 Helps PSO balance **exploration and convergence**.

3️⃣ α / β / ρ range

```
α ∈ [0.5 , 1.5]
β ∈ [1 , 5]
ρ ∈ [0.5 , 1]
```

These are **ACO parameters**:

- **α** → pheromone importance
- **β** → heuristic (distance) importance
- **ρ** → pheromone evaporation rate

👉 PSO searches these ranges to find the **best combination**.

4️⃣ NUM_ANTS / ACO_ITERS (10 / 100)

- **10 ants** explore the network each iteration.
- ACO runs for **100 iterations**.

👉 More ants and iterations improve **path discovery**.

5️⃣ R_MAX / ΔR_MAX (150 / 5)

These detect **traffic congestion**.

- **R_MAX = 150** → road becomes congested
- **ΔR_MAX = 5** → sudden traffic increase

👉 If exceeded → **route re-planning is triggered**.

6️⃣ SIM_STEPS (20)

The simulation runs **20 time steps** for each route.

👉 At each step road conditions are updated.

7️⃣ COMBINED_ITERS / Threshold (5 / 5%)

Controls **combined route optimization**.

- Maximum **5 iterations**
- Stop if cost change < **5%**

👉 Means routes are **stable**.

Very Short Summary (For Viva)

- **PSO parameters** tune the ACO algorithm.
- **ACO parameters** control path searching by ants.
- **R_MAX and ΔR_MAX** detect congestion.
- **SIM_STEPS** simulate traffic changes.
- **COMBINED_ITERS** ensures network optimization stops when routes stabilize.

✅ If you want, I can also explain **why your algorithm specifically chose values like 30 particles, 10 ants, and 20 simulation steps (this is something examiners often ask in defense).**

6️⃣ SIM_STEPS (20)

The simulation runs 20 time steps for each route.

👉 At each step road conditions are updated. i dont understand these lines .ca n you explain me these .

No problem — this part can be confusing. Let's explain it **logically with a simple example**.

What SIM_STEPS = 20 Means

SIM_STEPS is the number of **time moments in the traffic simulation**.

It means the algorithm **observes and updates the road conditions 20 times while the vehicle travels.**

You can think of it like **20 time checkpoints**.

Simple Idea

```
SIM_STEPS = 20
```

means

```
t = 1,2,3,...,20
```

At every **time step (t)** the algorithm:

1. Updates **road condition factor R(i,t)**
2. Checks if **traffic congestion occurs**
3. Decides if **rerouting is needed**

Example

Suppose the route is:

```
3 → 2 → 6 → 9
```

The vehicle starts traveling.

The algorithm simulates traffic over time:

Time Step	What Happens
t=1	vehicle moves, road conditions updated

Time Step	What Happens
t=2	traffic slightly increases
t=3	congestion appears
t=4	rerouting may be triggered
...	simulation continues
t=20	simulation ends

So the system **checks road conditions 20 times during the trip**.

Why This Is Needed

Because **traffic changes over time**.

Example:

```
t=1  → road free
t=5  → moderate traffic
t=10 → heavy congestion
```

Without simulation steps, the algorithm would assume traffic is **constant**, which is unrealistic.

Simple Real-Life Analogy

Imagine **Google Maps**.

While driving:

- it checks traffic every few seconds
- if traffic increases → it suggests a new route

Your algorithm does the same thing but **20 times during the simulation**.

Short Answer (For Thesis / Viva)

> SIM_STEPS defines the number of time intervals used in the dynamic traffic simulation. At each step, road conditions are updated and congestion is checked, allowing the algorithm to perform rerouting if necessary.

✅ If you want, I can also explain **one deeper concept about SIM_STEPS that actually controls how realistic your traffic model is (most students miss this in their thesis).**

RESULTS AND ANALYSIS

A. Independent Optimization — All 17 Pairs

Table II shows the complete results. All 17 pairs reached their destination successfully (100%). A total of 7 dynamic reroutes occurred across all pairs. The average final route cost was 357.89. Pair P10 (5→9) achieved the lowest cost (194.94) via the direct edge. Pairs P2,P3, P4, P7, P8, P12, P15 each triggered one reroute due to mid-journey congestion spikes.

TABLE II – Independent Optimization: All 17 S-D Pairs

Pair	Src	Dst	Init Path	Final Path	Init Cost	Final Cost	Reroutes	Reached
P0	0	9	0→1→5→9	0→1→5→9	420.59	502.82	0	YES
P1	0	7	0→1→5→6→7	0→1→5→6→7	475.00	488.25	0	YES
P2	0	3	0→1→2→3	1→2→3	370.00	411.23	1	YES
P3	1	8	1→5→8	5→8	252.50	274.12	1	YES
P4	1	9	1→5→8→9	5→9	380.83	369.02	1	YES
P5	2	8	2→1→5→8	2→1→5→8	362.50	393.82	0	YES
P6	2	9	2→6→9	2→6→9	272.50	291.25	0	YES
P7	3	9	3→2→6→9	6→9	372.50	426.12	1	YES
P8	4	7	4→5→6→7	5→6→7	369.17	381.21	1	YES
P9	4	9	4→5→9	4→5→9	314.75	353.68	0	YES
P10	5	9	5→9	5→9	193.09	194.94	0	YES
P11	6	9	6→5→9	6→5→9	310.59	317.84	0	YES
P12	7	0	7→3→2→1→0	1→0	432.50	451.35	1	YES
P13	7	9	7→6→9	7→6→9	267.50	290.65	0	YES
P14	8	1	8→5→1	8→5→1	212.50	217.55	0	YES
P15	9	0	9→8→4→0	4→0	350.59	372.30	1	YES
P16	9	3	9→6→2→3	9→6→2→3	337.50	347.97	0	YES

here in (P2 0 3 0→1→2→3 1→2→3 370.00 411.23 1 YES) this row it tells that reroute occurs .and it occur only in node 1 ? node 2 and 3 does it occur or not? tell me in short and logically

Short logical explanation:

For **P2 (0 → 3)**

Initial path:

```
0 → 1 → 2 → 3
```

Final path:

```
1 → 2 → 3
```

Where did reroute happen?

The reroute happened **at node 1**.

Reason:

- The vehicle started at **node 0**
- When it reached **node 1**, congestion was detected
- The algorithm **replanned from the current node (1)**

So the new route becomes:

```
1 → 2 → 3
```

What about nodes 2 and 3?

- **Node 2** → no reroute occurred
- **Node 3** → destination reached, so no reroute

Logical summary

Node	What happened
0	start

Node	What happened
1	reroute triggered
2	normal traversal
3	destination

✅ **Short answer:**
The reroute occurred **only at node 1**, because congestion was detected there. Nodes **2 and 3 did not trigger rerouting**.

> why if (reroute occur or not) cost
> increases ? tell me

Short logical explanation:

Why cost increases even when reroute occurs (or does not occur)

The **cost in your algorithm depends on the road condition factor** R, not only on distance.

$$R_i(t) = d_i \times (1 + C_i)$$

Where

- d_i = distance
- C_i = congestion factor

1️⃣ Traffic conditions change during simulation

During the **20 simulation steps**, congestion can increase on roads.

So even if the **route stays the same**, the **road condition value** R becomes larger → **cost increases**.

2️⃣ Rerouting may choose a longer path

When congestion is detected, the algorithm may select a **different path** that avoids traffic but is **slightly longer**.

Example:

```
Original path distance = shorter
New path distance = slightly longer but less congested
```

This can increase total cost.

3️⃣ Dynamic road penalties

Your algorithm continuously updates:

```
R_i(t)
```

So the **final cost reflects updated traffic conditions**, not just the initial shortest distance.

✅ **Very short answer (for viva):**

> The route cost increases because road condition factors change during the dynamic simulation. Congestion increases edge weights, so even if the route remains the same or rerouting occurs, the final cost can become higher.

i dont understand . can you tellme with a short example

Let's use a **very small example** so it becomes clear.

Example Network

```
0 ----1---- 1 ----1---- 2 ----1---- 3
```

Each road distance = **1**

Initial route:

```
0 → 1 → 2 → 3
```

Initial cost:

```
1 + 1 + 1 = 3
```

During Simulation (Traffic Appears)

Suppose traffic increases on **road 1 → 2**.

Now its cost becomes:

```
distance = 1
traffic penalty = +1
```

So new cost:

```
1 → 2 = 2
```

New Route Cost

Now total cost becomes:

```
0→1 = 1
1→2 = 2
2→3 = 1
```

Total:

```
1 + 2 + 1 = 4
```

So the **route is the same**, but **cost increased** because traffic increased.

Key Idea

Cost is not just distance.

```
Cost = distance + congestion
```

So when traffic changes during simulation, **cost increases**.

✅ One-line answer

> Cost increases because traffic congestion increases the road weight during simulation, even if the route does not change.

C. Combined Route Optimization — Convergence and Comparison

The combined loop converged after 3 **iterations** with average costs of 356.60 → 387.43 → 370.40 (final change = 4.39%, below the 5% threshold). Table IV compares per-pair costs. Of 17 pairs, **4 improved** under combined mode (most notably P11: 6→9 improved by 41.8%, switching from 6→5→9 to the direct edge 6→9 once bottleneck penalties redirected traffic) and **13 increased slightly** — correctly reflecting aggregate traffic load on shared segments that independent optimization did not account for. Routes remained unchanged for 10 of 17 pairs.

TABLE IV – Per-Pair Comparison: Independent vs. Combined

Pair	Src	Dst	Indep Cost	Comb Cost	Δ%	Same Route?	Combined Path
P0	0	9	502.82	513.94	+2.2%	NO	1→5→9
P1	0	7	488.25	509.05	+4.3%	YES	0→1→5→6→7
P2	0	3	411.23	445.68	+8.4%	NO	2→3
P3	1	8	274.12	446.60	+62.9%	NO	1→0→4→8
P4	1	9	369.02	360.82	-2.2%	NO	1→5→9
P5	2	8	393.82	435.70	+10.6%	NO	2→6→5→8
P6	2	9	291.25	309.56	+6.3%	YES	2→6→9
P7	3	9	426.12	432.91	+1.6%	YES	6→9
P8	4	7	381.21	364.12	-4.5%	YES	5→6→7
P9	4	9	353.68	368.13	+4.1%	YES	4→5→9
P10	5	9	194.94	203.24	+4.3%	YES	5→9
P11	6	9	317.84	184.92	-41.8%	NO	6→9
P12	7	0	451.35	464.12	+2.8%	YES	1→0
P13	7	9	290.65	302.02	+3.9%	YES	7→6→9
P14	8	1	217.55	214.65	-1.3%	YES	8→5→1
P15	9	0	372.30	380.97	+2.3%	YES	4→0
P16	9	3	347.97	360.44	+3.6%	NO	9→6→7→3

can oyu tell me (The combined loop converged after 3 iterations with average costs of 356.60 → 387.43 → 370.40 (final change = 4.39%, below the 5% threshold). Table IV compares per-pair costs. Of 17 pairs, 4 improved under combined mode (most notably P11: 6→9 improved by 41.8%, switching from 6→5→9 to the direct edge 6→9 once bottleneck penalties redirected traffic) and 13 increased slightly — correctly reflecting aggregate traffic load on shared segments that independent optimization did not account for. Routes remained unchanged for 10 of 17 pairs.) this partin easy and logically and in short ..i dont understand it .....

Here is the **same paragraph explained very simply and logically**.

1️⃣ What “converged after 3 iterations” means

The combined optimization **recalculates routes multiple times** because routes affect each other.

Average cost changed like this:

```
Iteration 1 → 356.60
Iteration 2 → 387.43
Iteration 3 → 370.40
```

After the **3rd iteration**, the change in cost became **4.39%**, which is **less than the stopping limit (5%)**.

So the algorithm **stopped because the results became stable**.

2️⃣ What happened to the 17 routes

You had **17 source–destination pairs**.

After combined optimization:

- **4 routes improved (lower cost)**
- **13 routes increased slightly**
- **10 routes stayed the same path**

3️⃣ Why some routes improved

Example **P11 (6 → 9)**

Before:

```
6 → 5 → 9
```

After combined optimization:

```
6 → 9
```

Because **road 5→9 was heavily used by many routes**, the algorithm added congestion penalties. This pushed the algorithm to choose the **direct road 6→9**, which reduced cost by **41.8%**.

4️⃣ Why many costs increased

Some routes became slightly more expensive because:

- Many vehicles were using the **same roads**
- The algorithm added **traffic penalties**

This makes the system **more realistic**, since shared roads become congested.

✅ Very short summary

> The combined optimization ran 3 times until route costs stabilized. Out of 17 routes, 4 improved, 13 increased slightly due to shared-road congestion penalties, and 10 routes kept the same path.

If you want, I can also explain **one very important concept hidden in this table (why independent optimization gives lower cost but less realistic traffic results)**.

P0 0 9 502.82 513.94 +2.2% NO 1→5→9 can you show me the calculation of % .i doint

undersrtand it

Let's calculate the **percentage change step-by-step** for your row:

```
Independent Cost = 502.82
Combined Cost = 513.94
```

Step 1 — Find the difference

$$\text{Difference} = 513.94 - 502.82$$

```
Difference = 11.12
```

Step 2 — Divide by the original value (Independent Cost)

$$\frac{11.12}{502.82}$$

```
= 0.0221
```

Step 3 — Convert to percentage

$$0.0221 \times 100$$

```
= 2.21%
```

Rounded:

```
≈ +2.2%
```

✅ Final Result

```
Δ% = ((Combined Cost − Independent Cost) / Independent Cost) × 100

Δ% = ((513.94 − 502.82) / 502.82) × 100
Δ% = 2.2%
```

Meaning

The combined optimization made the route **2.2% more expensive** because shared-road congestion penalties increased the cost.

The combined optimization increased average cost by 3.5% (357.89 → 370.40). from where they calculate 357.89 and 370.40 ? can you told me these thing in short and easy and in logical way

Those two numbers are simply the **average cost of all 17 routes** in two different cases.

- **357.89** → average cost when routes are optimized **independently**
- **370.40** → average cost after **combined optimization**

How they calculated it (logic)

Step 1: Add all Independent Costs (from the table)

Example:

```
502.82 + 488.25 + 411.23 + ... + 347.97
```

Total ≈ **6084.13**

Step 2: Divide by number of pairs

You have **17 S-D pairs**

```
6084.13 ÷ 17 = 357.89
```

So:

```
Average Independent Cost = 357.89
```

Step 3: Do the same for Combined Costs

Add all combined costs:

```
513.94 + 509.05 + 445.68 + ... + 360.44
```

Total ≈ **6296.80**

Then divide:

```
6296.80 ÷ 17 = 370.40
```

So:

```
Average Combined Cost = 370.40
```

Why cost increased

Because combined optimization adds **traffic penalties on shared roads**, making routes more realistic.

✅ **Short answer (for viva):**

> 357.89 and 370.40 are the averages of all 17 route costs. They are calculated by summing the costs of all pairs and dividing by 17 for independent and combined optimization respectively.

D. Overall Summary

Table V summarizes the overall performance. The combined optimization increased average cost by 3.5% (357.89 → 370.40). This is expected: independent mode under-estimates congestion on shared edges by ignoring other routes. Combined mode corrects this, producing more realistic traffic-aware routes at a modest cost premium. The network became more balanced overall, with fewer overloaded edges.

TABLE V – Overall Performance Summary

Metric	Independent	Combined
Total S-D Pairs / Success	17 / 17 (100%)	17 / 17 (100%)
Average Route Cost	357.89	370.40 (+3.5%)
Total Dynamic Reroutes	7	7
Shared Edges (2+ routes)	8	7
Critical Nodes (3+ routes)	8	9
Pairs with Lower Cost in this mode	—	4 / 17
Convergence Iterations	—	3 (4.39% < 5%)

from here i dont understand (Total Dynamic Reroutes 7 7) this part

Here is the **short and logical meaning** of **"Total Dynamic Reroutes = 7 (Independent) and 7 (Combined)"**.

What "Dynamic Reroute" means

A **dynamic reroute** happens when the algorithm **changes the route during simulation** because traffic congestion is detected.

This happens when:

```
R_i > R_MAX  or  ΔR_i > ΔR_MAX
```

Then the algorithm **recalculates the path from the current node**.

Why the value is 7

From **Table II**, these pairs triggered rerouting:

```
P2
P3
P4
P7
P8
P12
P15
```

So total:

```
7 reroute events
```

Why it is 7 in both Independent and Combined

Even after combined optimization, the **same number of congestion events occurred during simulation**, so the number of reroutes remained **7**.

The **routes or costs may change**, but the **number of rerouting events stayed the same**.

✅ **Very short answer (for viva):**

> "Total Dynamic Reroutes = 7" means that during the simulation, congestion triggered route recalculation **7 times across all 17 source-destination pairs. The same number appears in**

both modes because the same pairs experienced congestion events.

Convergence Iterations — 3 (4.39% < 5%) tell me why 3 is occured?

Short logical explanation:

What Convergence Iterations = 3 (4.39% < 5%) means

The **combined optimization loop keeps running until the route costs become stable**.

Stopping rule in your algorithm:

```
Stop if cost change < 5%
OR
maximum 5 iterations reached
```

What happened in your case

The average cost changed like this:

Iteration	Average Cost
1	356.60
2	387.43
3	370.40

Now the **change between iteration 2 and 3** is calculated.

$$\frac{|370.40 - 387.43|}{387.43} \times 100$$
$$= 4.39\%$$

Since

```
4.39% < 5%
```

the algorithm **stops**.

Why exactly 3 iterations occurred

Because the **cost stabilized after the 3rd iteration**, meeting the stopping condition.

The algorithm did **not need more iterations**.

✅ **Very short answer (for viva):**

> The combined optimization stopped after 3 iterations because the change in average route cost became 4.39%, which is below the 5% convergence threshold.

does this 3 iteration needed for all route ?

Short answer: **Yes — but not individually.**

Logical explanation

The **3 iterations belong to the combined optimization loop for the whole network**, not for each route separately.

Process:

1. **All 17 routes are optimized independently first.**
2. Then the **combined optimization loop starts.**
3. In each iteration, the algorithm:
    - checks **shared roads**
    - adds **traffic penalties**
    - **re-optimizes all 17 routes again**

So in your experiment:

```
Iteration 1 → optimize all 17 routes
Iteration 2 → re-optimize all 17 routes
Iteration 3 → re-optimize all 17 routes
```

After iteration 3, the **average cost change became 4.39% (<5%)**, so the algorithm stopped.

✅ Important point

The **3 iterations are for the whole network**, not per route.

So:

```
3 iterations × 17 routes re-optimized each time
```

✅ Very short viva answer

> The 3 iterations are part of the combined network optimization loop. In each iteration all 17 routes are recalculated until the average cost stabilizes below the 5% threshold.

C_i(t) += 0.3 × (freq_i − 1) for edges with freq_i ≥ 2 from where this formula is obtained ? tell me

Short and logical answer:

Where this formula comes from

$$C_i(t) += 0.3 \times (freq_i - 1)$$

This formula is **not taken from a fixed theory or standard PSO/ACO equation**.
It is a **design choice (penalty model) introduced in your algorithm** to simulate **traffic congestion on shared roads**.

Why this formula is used

- **$freq_i$** = number of routes using the same edge
- If many routes use the same road → congestion increases.

So the algorithm **adds a penalty to the congestion factor** C_i.

Example:

$freq_i$	penalty added
1	0 (no congestion)
2	0.3
3	0.6
4	0.9

So **more shared routes → higher cost for that road**.

Why ($freq_i$ − 1)

Because the **first route does not cause congestion**.
Congestion starts when **more than one route uses the edge**.

Why 0.3

0.3 is a **scaling factor chosen experimentally** to control how strongly congestion affects route cost.

✅ Short thesis explanation

> The congestion penalty function $C_i(t) += 0.3(freq_i - 1)$ is a heuristic designed to increase the cost of edges shared by multiple routes, thereby modeling traffic congestion effects in the combined optimization stage.

✅ Very short viva answer

> The formula is a heuristic congestion penalty added when multiple routes share the same edge, increasing its cost to simulate traffic competition.

Total S-D Pairs / Success 17 / 17 (100%) 17 / 17 (100%) why here 100% success is called here ? can you tell me in simple and easy and in short and logically

Simple explanation

Total S-D Pairs / Success = 17 / 17 (100%)

This means:

- There are **17 source–destination pairs** in the experiment.
- The algorithm **successfully found a path for all 17 pairs**.
- None of the routes failed.

So the success rate is:

$$\text{Success Rate} = \frac{17}{17} \times 100 = 100\%$$

Example

If it had been like this:

```
17 pairs tested
16 reached destination
1 failed
```

Then:

```
Success = 16/17 = 94%
```

But in your results:

```
17 reached destination
0 failed
```

So:

```
Success = 100%
```

✅ **Very short answer (for viva):**

> It is called **100% success** because the algorithm successfully found valid routes for all 17 source–destination pairs.